
Mess Congestion Live Dashboard

Web-D + CV

Context

We're building the **Web Development + Computer Vision** portion of a college project: a live dashboard showing mess (dining hall) congestion — queue length, seat occupancy, headcount, and crowd level — so students can decide when to eat.

. We only own: **CV inference, backend API, database, and the dashboard frontend**. Assume the Pi units already POST JPEG images to our ingestion endpoint every 10 seconds — just the receiving side.

Tech stack (all cloud-hosted, no server to manage)

- **CV inference:** Python, `ultralytics` (YOLOv8n pretrained, COCO person class) — runs inside the backend service below, not on a separate machine.
- **Backend:** FastAPI (two logical services: Ingestion API and Read API — two routers in one app for MVP), deployed on **Render** or **Railway** (free tier gives a public HTTPS URL, auto-deploys from GitHub). Fly.io is a fine alternative if you want more compute headroom for YOLO inference.
- **Time-series storage:** **InfluxDB Cloud** (managed, free tier) instead of self-hosted InfluxDB — same client library, just point at the cloud URL + token instead of `localhost`.
- **Cache:** **Upstash Redis** (managed, serverless, free tier, REST-based — works well from Render/Railway without needing a persistent connection) instead of self-hosted Redis.
- **Dashboard:** **Streamlit Community Cloud** (free, deploys straight from the GitHub repo).
- **Local dev:** Docker Compose still useful for running InfluxDB + Redis *locally* while writing code, before pointing at the real cloud instances — but nothing in production runs on a machine we manage.

Concurrency caveat: Streamlit Community Cloud's free tier is not built for ~300 truly-concurrent live sessions — it's fine for dev/demo traffic, but the same risk from before applies, just one layer up: if load testing (see Scaling plan) shows it struggling, the fallback is the same — a lightweight static HTML/JS page polling the Read API, hosted for free on Render/Vercel/Netlify, sidesteps Streamlit's concurrency limits entirely since it's stateless per request.

Architecture

```
Pi cameras (external team)
  | HTTPS POST JPEG every 10s (public internet, not just campus LAN)
  v
[Ingestion API] -- Render/Railway -- receives image + camera_id,
  |               validates, hands to inference
  v
[YOLOv8 inference] -- runs inside the same service -- person detection,
  |               ROI zone logic -> queue count, seat occupancy,
  |               headcount, crowd level
  |
  +--> InfluxDB Cloud (every reading, timestamped -- trend charts / history)
  +--> Upstash Redis (overwrite "latest" key per camera -- fast dashboard reads)

[Read API] -- same Render/Railway service -- GET endpoints, reads ONLY
  |               from Upstash Redis (current) and InfluxDB Cloud (history)
  v
[Streamlit dashboard] -- Streamlit Community Cloud -- polls Read API
  |               every ~8-10s via st.cache_data(ttl=5) + autorefresh;
  |               never touches InfluxDB/Redis directly
  v
Students (up to ~300 concurrent)
```

Key design rule: the dashboard never queries the database directly. All reads go through the Read API, which serves cached data from Upstash Redis. This is what lets the read side scale independently of database load — and it also means the Read API service is the only thing that needs to scale if traffic spikes, not the databases.

1 Repo structure

Set up:

```
mess-dashboard/
|-- inference/
|   |-- model.py           # YOLOv8 wrapper: load model, run detection
|   |-- zones.py           # ROI zone definitions + point-in-polygon logic
|   |-- metrics.py         # raw detections + zones -> queue/seat/headcount/crowd
|   '-- config/
|       '-- zones.json     # per-camera zone polygons (editable, no code change)
|-- api/
|   |-- main.py            # FastAPI app, mounts both routers
|   |-- ingestion.py       # POST /ingest/{camera_id}
|   |-- read.py            # GET /status, /status/{camera_id}, /history/{camera_id}
|   |-- storage/
|       |-- influx_client.py # points at InfluxDB Cloud URL + token (env vars)
|       '-- redis_client.py  # points at Upstash Redis URL + token (env vars)
|   '-- models.py          # pydantic schemas for responses
|-- dashboard/
|   '-- app.py             # Streamlit app (deployed on Streamlit Community Cloud)
|-- tests/
```

```
| |-- sample_images/          # a few mess-hall test images for local dev
| '-- test_metrics.py
|-- docker-compose.yml       # InfluxDB + Redis for LOCAL DEV ONLY
|-- requirements.txt
|-- .env.example              # INFLUX_URL, INFLUX_TOKEN, REDIS_URL, REDIS_TOKEN
'-- README.md
```

2 CV inference module

Build `inference/model.py`:

- Load `yolov8n.pt` via `ultralytics.YOLO`, run inference on an input image, filter detections to `class == 0` (person), return list of bounding boxes + confidence scores.
- Confidence threshold configurable (default 0.4 — will be tuned once real cameras are mounted).

Build `inference/zones.py`:

- Load per-camera zone polygons from `config/zones.json`. Format: `{"camera_1": {"queue": [[x,y],...], "seats": [{"id": "seat_1", "polygon": [[x,y],...}], ...}}}`.
- Point-in-polygon check (use `shapely` or a simple ray-casting function) to test whether a detection's bbox centroid falls inside a given zone.

Build `inference/metrics.py`:

- `compute_metrics(detections, zones) -> dict` returning:

```
{
  "headcount": <total person count in frame>,
  "queue_count": <persons whose centroid is inside the queue zone>,
  "seats": {"seat_1": true, "seat_2": false, ...},
  "seats_occupied": <count>,
  "seats_total": <len(seat zones)>,
  "crowd_level": "low" | "medium" | "high"
  // thresholds: <TODO set based on mess capacity>
}
```

Crowd level thresholds should be configurable constants at the top of the file, not hardcoded magic numbers — tune these against the mess's actual capacity once known.

Write `tests/test_metrics.py` with a couple of sample images (mock/dummy zone configs) verifying the pipeline runs end-to-end without needing a real camera feed.

Note: seat occupancy is a heuristic (person-bbox-in-seat-zone), not a trained classifier. Flag this clearly in code comments so it's easy to swap out for a custom-trained model later without touching the API layer.

3 Ingestion API

Build `api/ingestion.py`:

-
- `POST /ingest/{camera_id}` — accepts multipart form image upload (JPEG). Validate: content-type is image, `camera_id` is a known/expected value.
 - On receipt: run inference synchronously (fine for 2 cameras every 10s — no task queue needed yet, but structure the function so it *could* be moved behind Celery/RQ later without changing the endpoint contract).
 - Write the full metrics dict + timestamp to InfluxDB Cloud (one measurement per camera per reading).
 - Overwrite the "latest" key for that camera in Upstash Redis (key like `latest:{camera_id}`, value = JSON metrics dict, TTL of e.g. 30s so stale-camera data doesn't linger if a Pi goes offline).
 - Return `{"status": "ok", "metrics": {...}}`.

Handle the case where inference fails or the image is corrupt gracefully — log and return a 4xx, don't crash the service.

Credentials (InfluxDB token, Upstash token) come from environment variables set in Render/Railway's dashboard — never hardcoded, never committed. Keep a `.env.example` in the repo showing the variable names without real values.

4 Read API

Build `api/read.py`:

- `GET /status` — returns latest metrics for **all** cameras (reads all `latest:*` keys from Upstash Redis in one call). This is the main endpoint the dashboard hits.
- `GET /status/{camera_id}` — latest metrics for one camera.
- `GET /history/{camera_id}?minutes=60` — queries InfluxDB Cloud for that camera's readings over the given window, downsampled if needed, for a trend chart.
- These endpoints must **never** run YOLO inference or touch the Pi units — read-only, cache/DB reads only.
- Add basic response caching at the FastAPI level too (e.g. a short in-process TTL cache) as a second layer of protection if Upstash rate limits get hit on the free tier.

5 Streamlit dashboard

Build `dashboard/app.py`:

- Fetch `/status` from the Read API (the Render/Railway public URL), wrapped in `st.cache_data(ttl=5)` so it's not re-fetched on every Streamlit internal rerun.
- Use `streamlit-autorefresh` (`st.autorefresh(interval=8000, key="refresh")`) to refresh roughly on the same cadence as the capture interval — don't poll faster than the data changes.
- Show per-camera: current crowd level (color-coded — green/amber/red), headcount, queue count, seats occupied/total.
- Show a simple trend chart (last hour) using `/history/{camera_id}` — line chart of headcount or crowd level over time, so students can see "is it getting worse or better."

-
- If a camera's data is missing/stale (Redis TTL expired), show a clear "offline" state rather than silently showing 0.
 - Keep the whole app read-only — no write actions from the dashboard.
 - Store the Read API's base URL in Streamlit Cloud's secrets manager (`st.secrets`), not hard-coded, so it's easy to repoint if the backend URL changes.

6 Local dev environment

`docker-compose.yml` spins up InfluxDB and Redis **for local development only** — write and test code against these locally, then point the deployed Render/Railway service at the real InfluxDB Cloud + Upstash instances via environment variables. Don't try to make local dev hit the cloud services directly; that burns free-tier quota and adds network latency to every test run.

`requirements.txt` should pin: `fastapi`, `uvicorn`, `ultralytics`, `influxdb-client`, `redis` (or `upstash-redis` if using their REST client), `streamlit`, `streamlit-autorefresh`, `shapely`, `python-multipart`, `pydantic`.

Write a `README.md` with: setup steps, how to run `docker-compose up` for local dev, how to start the API locally (`uvicorn api.main:app -reload`), how to start the dashboard locally (`streamlit run dashboard/app.py`), how to POST a test image to the ingestion endpoint with `curl`, and how to set up the Render/Railway + InfluxDB Cloud + Upstash + Streamlit Cloud accounts and connect them (env vars / secrets needed in each).

7 Scaling plan (light-touch given the timeline)

There isn't room to over-build this, but the caching split (Read API → Redis only, never the DB) already does most of the work for free. On top of that:

1. **Streamlit statelessness:** the app should have zero server-side state that would break if Streamlit Cloud spins up multiple instances or restarts (no global mutable variables, no in-memory storage beyond `st.session_state`). This costs nothing extra to do correctly from the start, so don't skip it even under time pressure.
2. **One load test, once:** a bare-bones `locust` or `k6` script hitting the deployed `/status` endpoint repeatedly — not something to build carefully, just enough to know the Read API survives ~300 rps *before* launch day, not during it. Also worth checking Render/Railway's free-tier rate limits and Upstash's free-tier request quota against this number early — free tiers are the most likely bottleneck, not your code.
3. **Backend scaling:** if the Render/Railway free tier can't handle the ingestion + read load together, split them into two separate deployed services (they're already separate routers, so this is a config change) so a burst of dashboard reads can't starve image ingestion.
4. **Fallback, documented not built:** if Streamlit Cloud still can't hold up under ~300 concurrent users, the Read API is a clean REST interface — a static HTML/JS polling page (hosted free on Vercel/Netlify/Render) could replace it without touching the backend. Write this down in the `README` as a contingency; don't spend time building it unless the load test forces your hand.